CMPSC 465 — Homework 10
Dynamic Programming

Mohammad Abu Shariha

November 15, 2025

Problem 1: Contiguous Subsequences

Task. Given a sequence $a_1, a_2, \ldots, a_n$, find the contiguous subsequence with maximum possible sum. (The empty subsequence has sum 0.)

(i) Subproblems

Define:

$$dp[i] = \text{maximum sum of a contiguous subsequence ending at index } i.$$

(ii) Recurrence Relation

The subsequence ending at i either:

- extends the sequence ending at $i-1$, or
- starts at i.

$$dp[i] = \max(a_i,\ dp[i-1] + a_i).$$

Final answer:

$$\boxed{\max\left(0,\ \max_{1 \le i \le n} dp[i]\right)}$$

Justification. This ensures that if all elements are negative, the empty subsequence (sum 0) is chosen.

(iii) Base Case

$$dp[1] = a_1.$$

(iv) Runtime Analysis

A single pass computes all values:

$$O(n) \text{ time, } O(1) \text{ space if optimized.}$$

Problem 2: Zig-Zag Sequence

Task. Find the length of the longest subsequence that strictly increases and then strictly decreases.

(i) Subproblems

$inc[i]$ = length of the longest strictly increasing subsequence ending at i,

$dec[i]$ = length of the longest strictly decreasing subsequence starting at i.

A zig-zag subsequence peaks at k, giving:

$$zigzag(k) = inc[k] + dec[k] - 1.$$

(ii) Recurrence Relation

Increasing:

$$inc[i] = 1 + \max_{j<i,\ a_j<a_i} inc[j].$$

Decreasing:

$$dec[i] = 1 + \max_{j>i,\ a_j<a_i} dec[j].$$

Answer:

$$\max_{1\le k\le n} (inc[k] + dec[k] - 1).$$

Justification. Every valid zig-zag strictly increases then strictly decreases. Computing LIS and LDS around each peak covers all possibilities.

(iii) Base Cases

$$inc[i] = 1, \qquad dec[i] = 1.$$

(iv) Runtime Analysis

Two nested loops:

$$O(n^2) \text{ time.}$$

Problem 3: Palindromic Subsequences

Task. Compute the length of the longest palindromic subsequence (LPS) of $a[1..n]$.

(i) Subproblems

$$dp[i][j] = \text{length of the longest palindromic subsequence in } a[i..j].$$

(ii) Recurrence Relation

When endpoints match:

$$dp[i][j] = 2 + dp[i+1][j-1].$$

When they do not:

$$dp[i][j] = \max(dp[i+1][j],\ dp[i][j-1]).$$

Justification. Matched characters form the two ends of a palindrome; otherwise, one must be excluded.

(iii) Base Cases

$$dp[i][i] = 1 \quad \text{(single character)}.$$

$$dp[i][j] = 0 \quad \text{if } i > j.$$

(This is required when computing $dp[i+1][j-1]$ for 2-character palindromes.)

(iv) Runtime Analysis

$$O(n^2) \text{ time.}$$

Problem 4: Longest Common Substring

Task. Given strings $x = x_1 \dots x_n$ and $y = y_1 \dots y_m$, find the length of the longest common substring.

(i) Subproblems

$$dp[i][j] = \text{length of the longest common substring ending at } (i, j).$$

(ii) Recurrence Relation

If characters match:

$$dp[i][j] = dp[i-1][j-1] + 1.$$

Else:

$$dp[i][j] = 0.$$

Final answer:

$$\max_{i,j} dp[i][j].$$

Justification. Substrings require contiguity, so mismatches reset to zero.

(iii) Base Cases

$$dp[i][0] = dp[0][j] = 0.$$

(iv) Runtime Analysis

$O(nm)$ time.

Problem 5: Gold Bars

Task. Given prices $p_1, p_2, \ldots, p_n$ for pieces of gold of lengths 1 to n, compute the maximum obtainable value by cutting a bar of length n.

(i) Subproblems

$$dp[i] = \text{maximum obtainable value from length } i.$$

(ii) Recurrence Relation

$$dp[i] = \max_{1 \le j \le i} \left(p_j + dp[i-j] \right).$$

Justification. Trying all possible first cuts ensures optimality; the remaining length is solved optimally by DP.

(iii) Base Case

$$dp[0] = 0.$$

(iv) Runtime Analysis

$$O(n^2) \text{ time.}$$